

7

Knowledge Representation

7.1 Introduction

In the preceding chapters, we have discussed problem-solving techniques using search graphs of solution paths; predicate logic for storing knowledge and inferencing new knowledge while solving problems requiring concepts of artificial intelligence (AI). We now proceed to familiarize ourselves with various other *knowledge representation* (KR) techniques. Knowledge representation is an important issue in both cognitive science and AI. In cognitive science, it is concerned with the way in which information is stored and processed by humans, while in AI, the main focus is on storing knowledge or information in such a manner that programs can process it and achieve human intelligence. Traditionally, knowledge representation was concerned with the ways in which knowledge was stored in an information system to accomplish a given task. In AI, *knowledge representation* is an important area because intelligent problem solving can be achieved and simplified by choosing an appropriate knowledge representation technique. Representing knowledge in some specific ways makes certain problems easier to solve. In computer science, particularly in AI, a number of representations have been devised to structure information. Since knowledge needs to be utilized to achieve intelligent behaviour, the fundamental goal of knowledge representation is to represent knowledge in a manner that facilitates the process of *inferencing* (i.e., drawing conclusions) from it.

Several programming languages oriented to KR have been developed till date. In Prolog (1972), knowledge is represented in the form of rules and facts as already explained in previous chapters. One can derive conclusions and prove theorems from known premises or facts. KL-ONE (1980s) was more specifically aimed at knowledge representation itself. The languages such as SGML, XML, RDF, etc., for handling electronic documents in web systems, have been developed in order to represent the structure of documents more explicitly. These languages facilitate processes such as

information retrieval, data mining, etc., which are also related to the field of KR. Any knowledge representation system should possess properties such as *learning*, *efficiency in acquisition*, *representational adequacy*, and *inferential adequacy*. These properties may be defined as follows:

- *Learning* refers to a capability that acquires new knowledge, behaviours, understanding, etc. It does not simply involve adding new facts to a knowledge base but new information may have to be *classified* to avoid redundancy and replication in the existing knowledge prior to storage to enable easy retrieval. There are a number of ways in which knowledge may be gained such as by reasoning and logic, by experience, by observing the world, by mathematical proofs, and by scientific methods.
- *Efficiency in acquisition* refers to the ability to acquire new knowledge using automatic methods wherever possible rather than relying on human intervention.
- *Representational adequacy* refers to the ability to represent the required knowledge.
- *Inferential adequacy* refers to the ability of manipulating knowledge to produce new knowledge from the existing one.

Efficiency of a method depends greatly on the representation scheme of the knowledge. Many AI methods have tried to model human intelligence. Knowledge representation is a core component of a number of applications such as expert systems, machine translation systems, computer-aided maintenance systems, information retrieval systems, and database front-ends. In expert systems, there are various ways of representing knowledge, namely, predicate logic, semantic networks, frames, conceptual dependency, etc., which have originated from theories of human information processing. In predicate logic, knowledge is represented in the form of rules and facts. An expert system that incorporates knowledge in the form of rules and facts is called a *rule-based expert system* (discussed in the next chapter). Other representation schemes offer the ability to represent structured knowledge regarding physical or conceptual objects more easily than rules.

In this chapter, we will introduce various knowledge representation techniques, namely, semantic network, extended semantic net and frame systems, with inheritance mechanism in each of these.

7.2 Approaches to Knowledge Representation

AI programs use structures known as knowledge structures to represent objects, facts, relationships, and procedures. The main function of these knowledge structures is to provide expertise and information so that a program can operate in an intelligent way. Knowledge structures are usually composed of both traditional and complex structures such as semantic network, frames, scripts, conceptual dependency structures, and so on. A knowledge base is a special type of database that holds knowledge of the domain. Knowledge bases share some of the functions of the database systems such as storing, updating, and retrieving information; however, they are much powerful and more structured. They are organized in hierarchical structures with in-built inheritance and inferencing capabilities. In the following subsections, we briefly survey some basic knowledge representation schemes.

7.2.1 Relational Knowledge

Relational knowledge comprises objects consisting of attributes and associated values. The simplest way of storing facts is to use a relational method. In this method, each fact is stored in a row of a relational table as done in relational database. A *table* is defined as a set of data values that is organized using a model of horizontal rows and vertical columns. The columns are identified by attribute names, while the rows are identified by the corresponding values appearing in a particular column subset. For example, a fact *John is a male whose age is 38 years and who is a graduate with a salary of Rs 20000* may be easily represented as one row in a table with attributes {Name, Age, Sex, Qualification, and Salary} (Table 7.1). This representation helps in storing the facts but gives little opportunity for inferencing; however, such structures can be used by inference engines.

Table 7.1 Relational Table

Name	Age (in years)	Sex	Qualification	Salary (in Rupees)
John	38	Male	Graduate	20000
Mike	25	Male	Undergraduate	15000
Mary	30	Female	Ph D	25000
James	29	Male	Graduate	18000

We can easily obtain the answers of the following queries (explicit) from Table 7.1:

- What is the age of John?
- How much does Mary earn?
- What is the qualification of Mike?

However, we cannot obtain answers for queries (implicit) such as *Does a person having a Ph D qualification earn more?* So, inferencing new knowledge is not possible from such structures.

7.2.2 Knowledge Represented as Logic

Inferential capability can be achieved if knowledge is represented in the form of formal logic. For example, knowledge regarding mortality such as *All humans are mortal* cannot be represented using relational approach; instead, it can be easily represented in predicate logic as follows:

$$(\forall X) \text{human}(X) \leftarrow \text{mortal}(X)$$

If we have a fact *John is human*, then we can easily infer that *John is mortal*. The advantages of this approach are that we can represent a set of rules, derive more facts, truths, and verify the correctness of new statements. This topic is dealt in detail in Chapter 4.

7.2.3 Procedural Knowledge

Procedural knowledge is encoded in the form of procedures which carry out specific tasks based on relevant knowledge. For example, an interpreter for a programming language interprets a program on the basis of the available knowledge regarding the syntax and semantics of the language. The advantages of this approach are that domain-specific knowledge can be easily represented and side effects of actions may also be modelled. However, there is a problem of completeness and consistency. *Completeness* means that all cases may not be represented, while *consistency* means that all deductions may not be correct.

We can extend the approaches mentioned above for storing knowledge in the knowledge base by organizing data into a hierarchical structure that allows inheritance mechanism. Knowledge organized in the form of semantic network and frames is referred to as *inheritable knowledge*.

7.3 Knowledge Representation using Semantic Network

The basic idea applied behind using a semantic network is that the meaning of a concept is derived from its relationship with other concepts, and that, the information is stored by interconnecting nodes with labelled arcs. For example, consider the following knowledge:

- Every human, animal, and birds are living things who can breathe and eat. All birds can fly. Every man and woman are human who have two legs. A cat has fur and is an animal. All animals have skin and can move. A giraffe is an animal and has long legs and is tall. A parrot is a bird and is green in colour. John is a man.

We can represent such knowledge using a structure called a *semantic network* (or semantic net, in short). It is conveniently represented in a graphical notation where nodes represent concepts or objects and arcs represent relation between two concepts. Alternatively, it can be described as a directed graph consisting of vertices representing concepts and edges that represent semantic relations between the concepts. Nodes can be easily added in this network. The graphical representation of this knowledge in semantic net is shown in Fig. 7.1.

In Fig. 7.1, we notice that the concepts or objects are related to each other by certain relations. The relations are represented in the figure by bold directed links. They may be defined as follows:

- **isa** This relation connects two classes (generic concepts), where one concept is a kind or subclass of the other concept. For example, Man isa Human, that is, Man is a subclass of the Human class.
- **inst** This relation relates specific members of a class, such as *John is an instance of Man*.

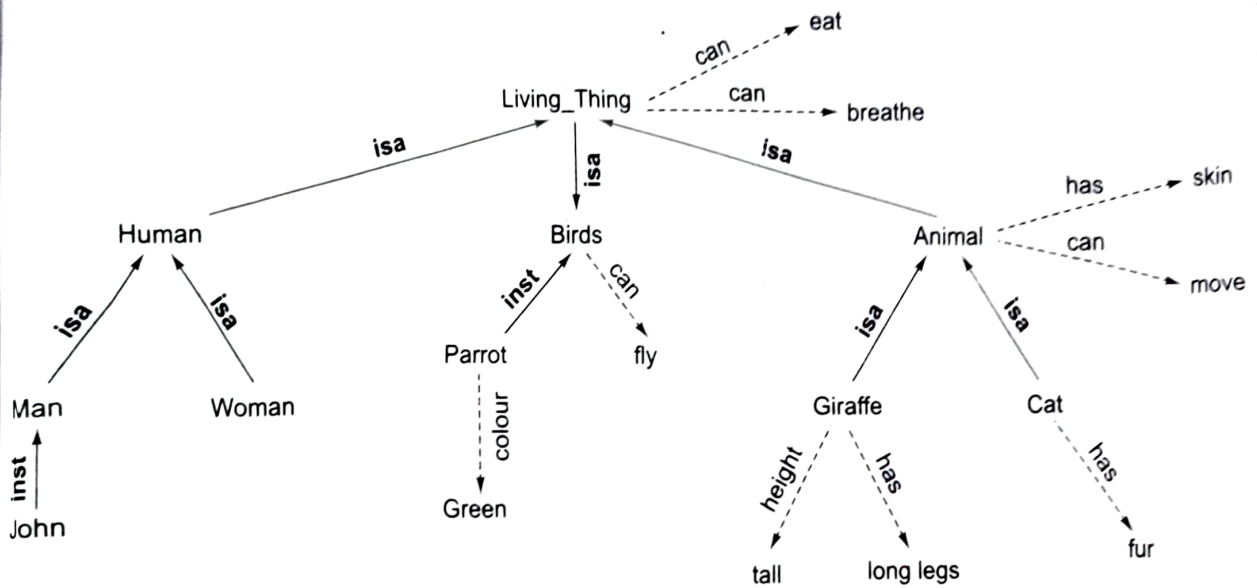


Figure 7.1 Knowledge Representation using Semantic Net

Other relations such as {can, has, colour, height} are known as *property relations*. These have been represented by dotted lines pointing from the concept to its property. In this structure, property inheritance is easily achieved. For example, the query *Does a parrot breathe?* can be easily answered as 'yes' even though this property is not associated directly with a parrot. It inherits this property from its super class named *Living_Thing*.

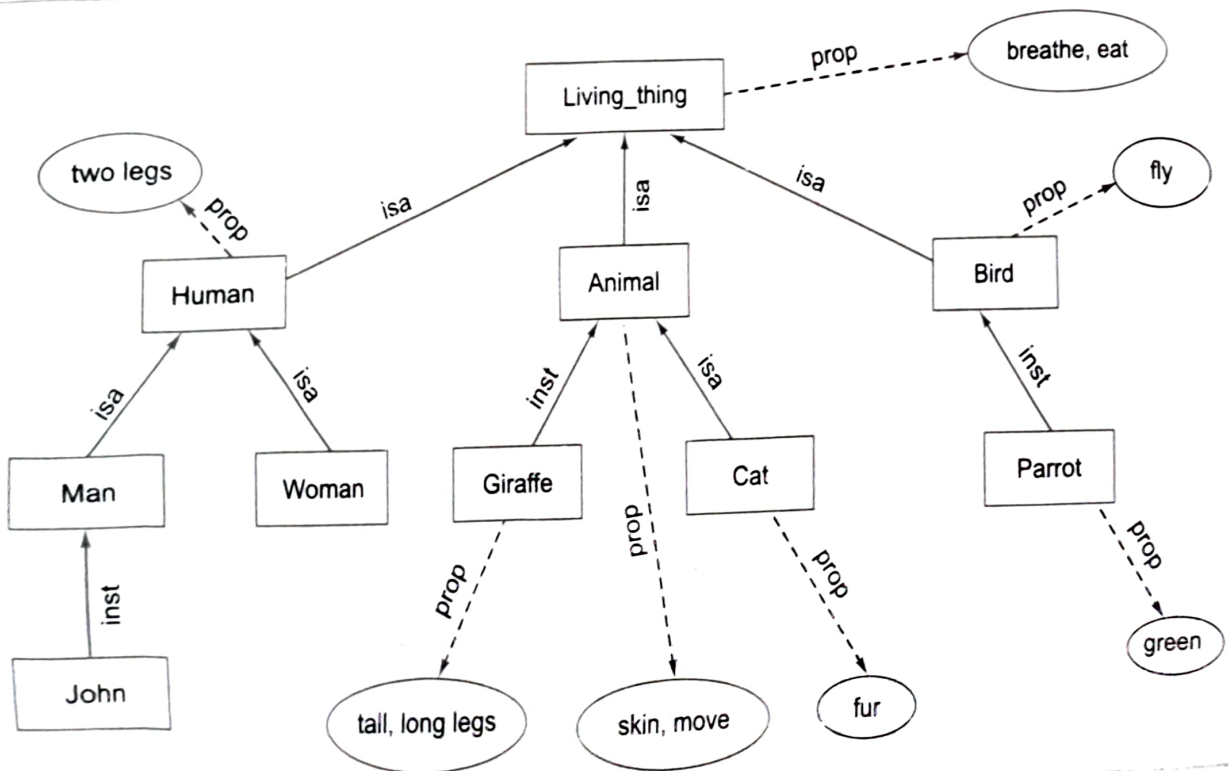


Figure 7.2 Concepts Connected with *prop* Links

We notice that *isa* and *inst* links have well-defined meaning, whereas other links which connect the attributes of class vary in their interpretations. The semantic net interpreter cannot understand all the attribute links unless their semantics are encoded into it. Since all the attributes of the class are the properties, we can designate a unique name to such relations, say, *prop* link. The interpretation of *prop* link will be simple that the attribute attached to a class is one of its properties. The information encoded above can be represented as a directed graph as shown in Fig. 7.2. In this figure, the square nodes represent concepts or objects connected with *isa* or *inst* links, whereas oval nodes represent property attributes attached to the square nodes. Property links are shown as dotted lines for more clarity.

7.3.1 Inheritance in Semantic Net

Hierarchical structure of knowledge representation allows knowledge to be stored at the highest possible level of abstraction which reduces the size of the knowledge base. Since semantic net is stored as hierarchical structure, the inheritance mechanism is in-built and facilitates inferencing of information associated with nodes in semantic net. It is a natural tool for representing taxonomically structured information and ensures that all the members and sub-concepts of a concept share common properties. It also helps us to maintain the consistency of the knowledge base by adding new concepts and members to existing ones. Properties attached to a particular concept can be easily inherited by all sub-concepts and their members. The algorithm to retrieve a property of an object (concept or instance) is given below

Algorithm 7.1 Property Inheritance Algorithm

Input: Object and property to be found from Semantic Net;
Output: returns yes, if the object has the desired property else returns false;
Procedure:

- Find an object in the semantic net;
- Found = false;
- While [(object $\neq$ root) OR Found] DO
 - {
 - If there is an attribute attached with an object then Found = true;
 - else {object = isa(object, class) or object = inst (object, class)}
 - };
- If Found = true then report 'Yes' else report 'No';

Semantic net can be implemented in any programming language along with an inheritance procedure implemented explicitly in that language. Prolog language is very convenient for representing an entire semantic structure in the form of facts (relation as predicate and nodes as arguments) and inheritance rules. Inheritance is easily achieved by unification of appropriate arguments in Prolog. The implementation of semantic net shown in Fig. 7.2 in Prolog is given as follows:

Prolog Facts

The facts in Prolog would be written as shown in Table 7.2.

Table 7.2 Prolog Facts

Isa facts	Instance facts	Property facts
<code>isa(living_thing, nil).</code>	<code>inst(john, man).</code>	<code>prop(breathe, living_thing)</code>
<code>isa(human, living_thing).</code>	<code>inst(giraffe, animal).</code>	<code>prop(eat, living_thing)</code>
<code>isa(animals, living_thing).</code>	<code>inst(parrot, bird)</code>	<code>prop(two_legs, human)</code>
<code>isa(birds, living_thing).</code>		<code>prop(skin, animal)</code>
<code>isa(man, human).</code>		<code>prop(move, animal).</code>
<code>isa(woman, human).</code>		<code>prop(fur, bird).</code>
<code>isa(cat, animal).</code>		<code>prop(tall, giraffe).</code>
		<code>prop(long_legs, giraffe).</code>
		<code>prop(tall, animal).</code>
		<code>prop(green, parrot).</code>

Inheritance Rules in Prolog

We know that in class hierarchy structure, a member subclass of a class is also a member of all super classes connected through *isa* link. For example, if *man* is a member of subclass *human*, then *man* is also member of *living_class*. Similarly, an instance of subclass is also an instance of all super classes connected by *isa* link. It should be noted that we cannot define an instance of an instance, that is, if *john* is an individual instance, then there cannot be another individual as an instance of *john*. Further, if *john* is an instance of *man*, then we can infer that *john* is *human* and also *living_thing*. Similarly, a property of a class can be inherited by lower sub-classes. A simple Prolog rule for handling inheritance in semantic net is given below. Various queries can be answered by the following inheritance program and are mentioned in Table 7.3.

Instance rules

```
instance(X, Y)      :-      inst(X, Y).
instance(X, Y)      :-      inst(X, Z), subclass(Z, Y).
```

Subclass rules

```
subclass(X, Y)      :-      isa(X, Y).
subclass(X, Y)      :-      isa(X, Z), subclass(Z, Y).
```

Property rules

```
property(X, Y)      :-      prop(X, Y).
property(X, Y)      :-      instance(Y, Z), property(X, Z).
property(X, Y)      :-      subclass(Y, Z), property(X, Z).
```

Table 7.3 Various Queries for Inheritance Program

English query	Prolog goal	Output
Is john human?	?- instance(john, humans).	Yes
Is parrot a living thing?	?- instance (parrot, living_thing).	Yes
Is giraffe an animal?	?- instance (giraffe, animal).	Yes
Is woman a subclass of living thing?	?- subclass(woman, living_thing).	Yes
Does parrot fly?	?- property(fly, parrot).	Yes
Does parrot have fur?	?- Does parrot have fur?	No
Does john breathe?	?- property (john, breathe).	Yes
Does cat fly?	?- property(fly, cat).	No

7.4 Extended Semantic Networks for KR

Logic and semantic networks are two different formalisms that can be used for knowledge representations. Simple semantic network is represented as a directed graph whose nodes represent concepts or objects and arcs represent relationships between concepts or objects. It can only express collections of variable-free assertions (Kowalski, 1979). The English sentences *john gives an apple to mike* and *john and mike are human* may be represented in semantic network as shown in Fig. 7.3, which shows alternative way of representing semantic net in detail with more semantic links.

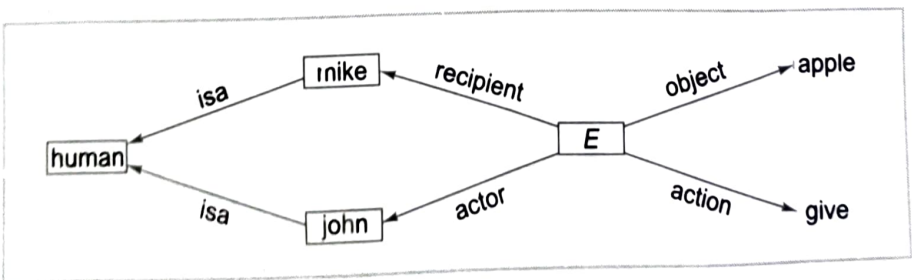


Figure 7.3 Semantic Net

Here, 'E' represents an event which is an act of giving, whose actor is *John*, the object is an *apple*, and recipient is *mike*. It should be noted that semantic net can hold semantic information about situation such as actor of an event *giving* is *john* and object is *apple*, in the sentence *John gives an apple to mike*. The relationships in the network shown in Fig. 7.3 can be expressed in clausal form of logic as follows:

- object (E, apple).
- action(E, give).
- actor (E, john).
- recipient(E, mike).
- isa (john, human).

Predicate relations corresponding to labels on the arcs of semantic networks always have two arguments. Therefore, the entire semantic net can be coded using binary representation (two-argument representation). Such representation is advantageous when additional information is added to the given system. For example, in the sentence *john gave an apple to mike in the kitchen*, it is easy to add *location(E, kitchen)* to the set of facts given above.

In first-order predicate logic, predicate relation can have n arguments, where $n \geq 1$. For example, the sentence *john gives an apple to mike* is easily represented in predicate logic by *give(john, mike, apple)*. Here, *john*, *mike*, and *apple* are arguments, while *give* represents a predicate relation. The predicate logic representation has greater advantages compared to semantic net representation as it can express general propositions, in addition to simple assertions. For example, the sentence *john gives an apple to everyone he likes* is expressed in predicate logic by the clause as follows:

$$\text{give}(\text{john}, X, \text{apple}) \leftarrow \text{likes}(\text{john}, X)$$

Here, the symbol X is a variable representing any individual. The arrow represents the logical connective *implied by*. The left side of $\leftarrow$ contains conclusion(s), while the right side contains condition(s).

Despite all the advantages, it is not convenient to add new information in an n -ary representation of predicate logic. For example, if we have 3-ary relationship *give(john, mike, apple)* representing *john gives an apple to mike* and to capture an additional information about kitchen in the sentence *john gave an apple to mike in the kitchen*, we would have to replace the 3-ary representation *give(john, mike, apple)* with a new 4-ary representation such as *give(john, mike, apple, kitchen)*.

Further, a clause in logic can have several conditions, all of which must hold for the conclusion to be true and can have several alternative conclusions, at least one of which must hold if all the conditions are true. For example,

- The sentence *if john gives something he likes to a person, then he also likes that person* can be expressed in clausal representation in logic as

$$\text{likes}(\text{john}, X) \leftarrow \text{give}(\text{john}, X, Y), \text{likes}(\text{john}, Y)$$

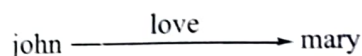
- The sentence *every human is either male or female* is expressed by the following clause

$$\text{male}(X), \text{female}(X) \leftarrow \text{human}(X)$$

In conventional semantic network, we cannot express clausal form of logic. To overcome this shortcoming, R Kowalski and his colleagues (1979) proposed an *extended semantic network* (ESNet) that combines the advantages of both logic and semantic networks. ESNet can be interpreted as a variant syntax for the clausal form of logic; it has the same expressive power as that of predicate logic with well-defined semantics, inference rules, and a procedural interpretation. It also incorporates the advantages of using binary relation as in semantic network rather than n -ary relations of logic. Therefore, ESNet is a much powerful representation as compared to logic and semantic network. In ESNet, the terms are represented by nodes similar to as done in conventional

semantic network; constant, variable, and functional terms are represented by *constant*, *variable*, and *functional nodes*, respectively.

Binary predicate symbols in clausal logic are represented by labels on arcs of ESNet. An *atom* of the form *love(john, mary)* is an arc labelled as *love* with its two end nodes representing *john* and *mary*. The direction of the arc (*link*) indicates the order of the arguments of the predicate symbol which labels the arc as follows:



Conclusions and *conditions* of clausal form are represented in ESNet by different kinds of arcs. The arcs denoting conditions (negative atoms) are drawn with dotted arrow lines. These are called *denial links* (----->), while the arcs denoting conclusions (positive atom) are drawn with continuous arrow lines. These are known as *assertion links* (——>). For example, the clausal representation (or rule) *grandfather(X, Y) ← father(X, Z), parent(Z, Y)* for grandfather in logic can be represented in ESNet as given in Fig. 7.4. Here, X and Y are variables; *grandfather(X, Y)* is the consequent (conclusion), and *father(X, Z)* and *parent(Z, Y)* are the antecedents (conditions).

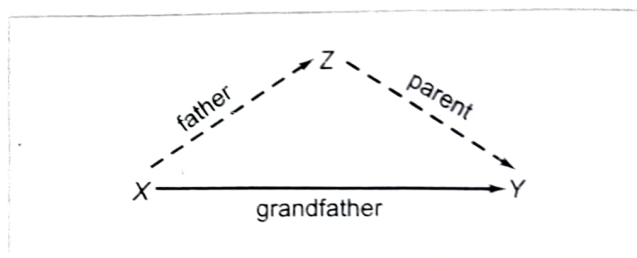


Figure 7.4 ESNet Representation

Similarly, the clausal rule *male(X), female(X) ← human(X)* can be represented using binary representation as *isa(X, male), isa(X, female) ← isa(X, human)* and subsequently in ESNet as shown in Fig. 7.5.

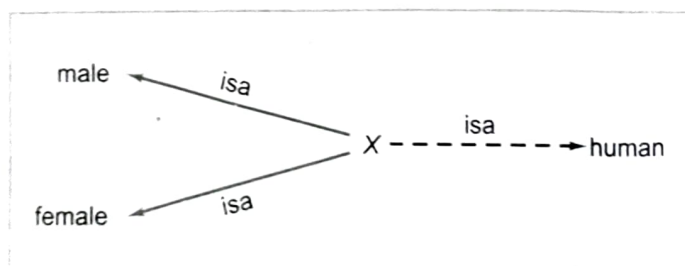


Figure 7.5 ESNet Representation

So, we have seen that any clause can be easily represented using ESNet which was not possible in ordinary Semantic Net. Now, we will discuss the inferencing rules associated with ESNet.

7.4.1 Inference Rules

Inference rules are embedded in the representation itself. Some of the inference rules are being discussed as follows:

- The representation of the inference for every action of *giving*, there is an action of *taking* in clausal logic is $action(f(X), take) \leftarrow action(X, give)$. The interpretation of this rule is that the event of *taking* action is a function of the event of *giving* action. In the ESNet representation, functional terms, such as $f(X)$, are represented by a single node. The representation of the statement $action(f(X), take) \leftarrow action(X, give)$ in ESNet is as shown in Fig. 7.6.

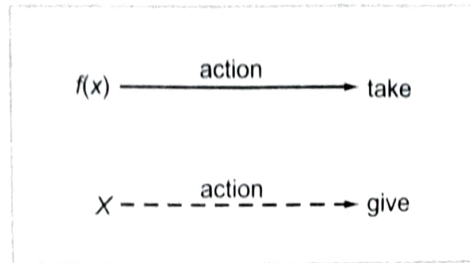


Figure 7.6 ESNet Representation

- The inference rule that an actor who performs a *taking* action is also the *recipient* of this action and can be easily represented in clausal logic; ESNet as given below. Here, E is a variable representing an event of an action of taking.

$$recipient(E, X) \leftarrow action(E, take), actor(E, X)$$

The representation of the above clause in ESNet is shown in Fig. 7.7.

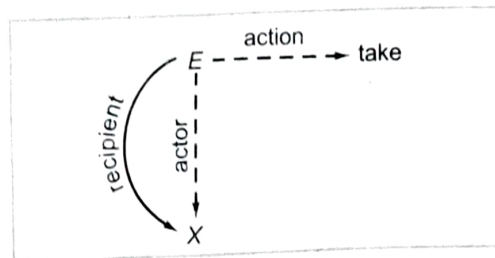


Figure 7.7 ESNet Representation

Example 7.1 Represent the following clauses in ESNet:

$recipient(E, X) \leftarrow action(E, take), actor(E, X)$
 $object(e, apple).$
 $action(e, take).$
 $actor(e, john).$

Solution Here, E is a variable for some event and 'e' is an actual event. ESNet representation of the clauses is shown in Fig. 7.8. Here, the partition is shown to delimit the clause pictorially by enclosing the clausal rule in an ellipse.

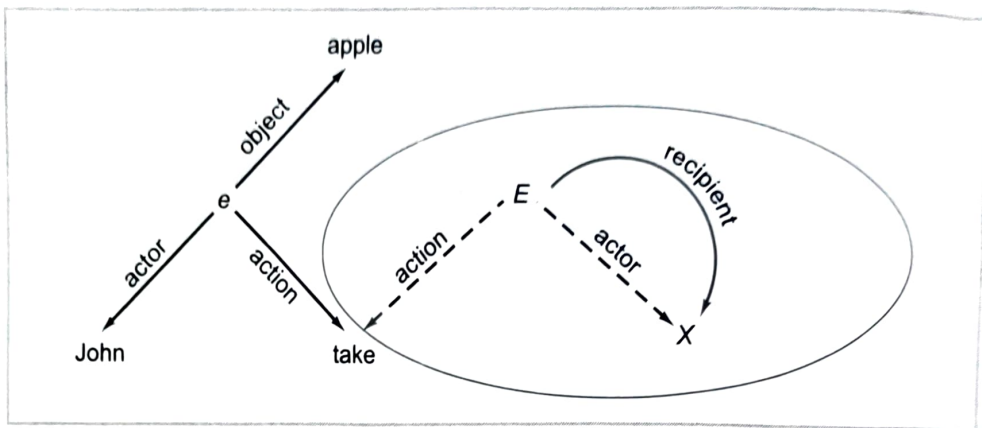
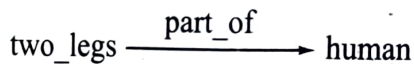


Figure 7.8 ESNet Representation

The hierarchy links, *isa*, *inst*, and *part_of* (or *prop*), are available in semantic networks and are also available as special cases in ESNet. It is important to note that *part-of* link has a hidden existential quantifier. For example, the assertion *every human has two legs* could be represented using *part_of* link as follows:



The interpretation of above representation is that *for every human, there exist two legs which are part of that human*.

- The contradiction in ESNet can be represented as shown in Fig. 7.9. Here *P part_of X* is conclusion and *P part_of Y* is condition, where *Y* is linked with *X* via *isa* link. Such kind of representation is contradictory and hence there is a contradiction in ESNet.

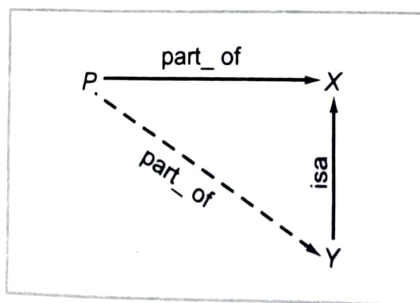


Figure 7.9 Contradiction in ESNet

7.4.2 Deduction in Extended Semantic Networks

Inference rules discussed in subsection 7.4.1 along with its semantics, form a part of ESNet. We know that in logic there are two types of inference mechanisms, namely

- forward reasoning inference mechanism
- backward reasoning inference mechanism

Forward reasoning inference mechanism (also called bottom-up approach) in clausal logic derives new assertion from old ones, that is, in this mechanism, we start with the given assertions and derive new assertions using clausal rule. On the other hand, in backward reasoning inference mechanism (or top-down approach), we prove the query from the set of clauses using resolution refutation method in clausal logic. Both these inference rules are available in ESNet also.

Forward Reasoning Inference

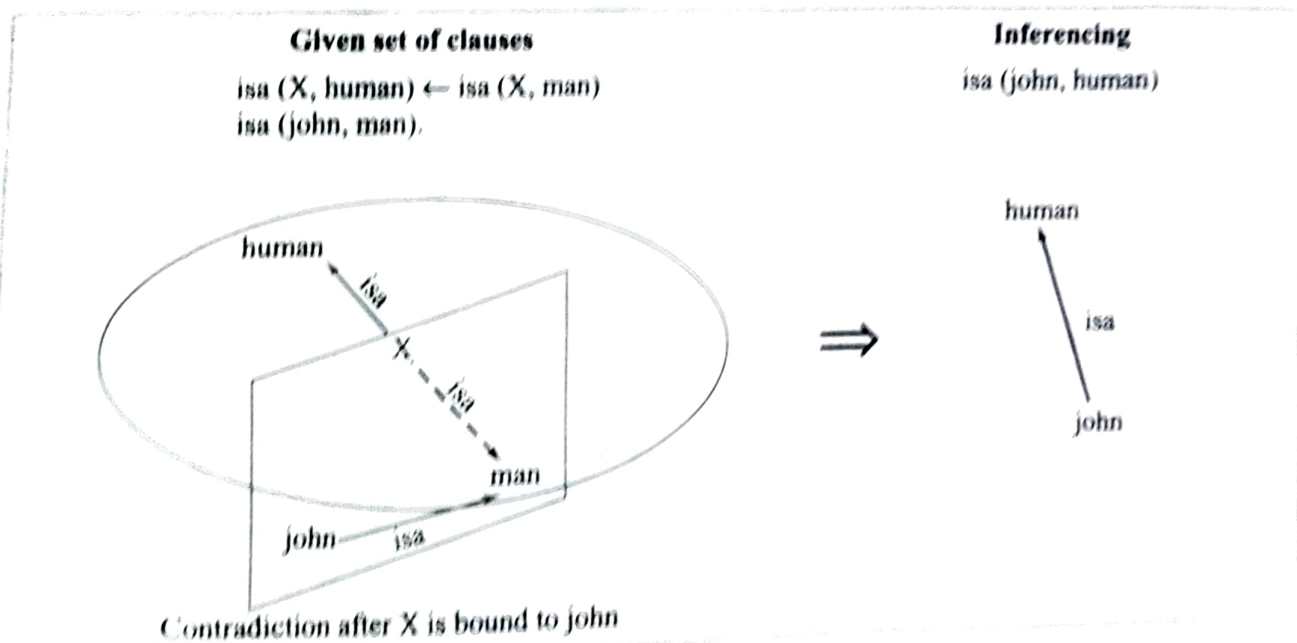
Given an ESNet, apply the following reduction (resolution) using modus ponens rule of logic {i.e., given $(A \leftarrow B)$ and B , then conclude A }. For example, consider the following set of clauses:

$isa(X, human) \leftarrow isa(X, man)$

$isa(john, man)$.

Using modus ponens rule of logic, we can easily derive that $isa(john, human)$ holds true. Now, let us derive or inference $isa(john, human)$ using ESNet representation. The clauses are represented in ESNet as shown Table 7.4. The new assertion $isa(john, human)$ is inferred by elimination of assertion and their denial links with the help of appropriate substitution. Here $isa(X, human)$ is an assertion link and $isa(X, man)$ is denial link.

Table 7.4 Forward Reasoning Inference Mechanism

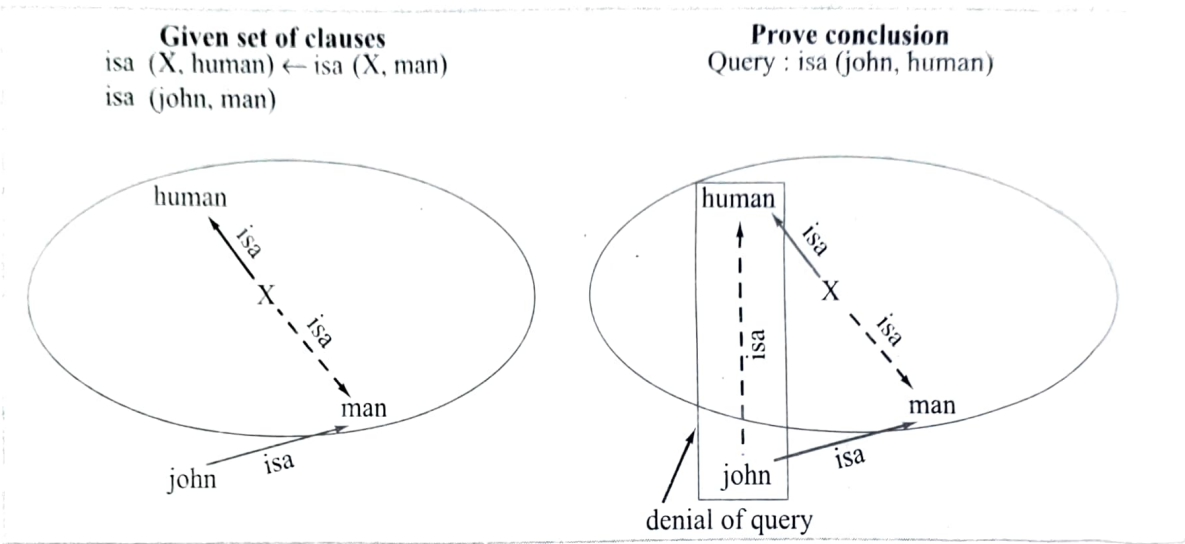


Here the contradiction is enclosed in a rectangular box.

Backward Reasoning Inference

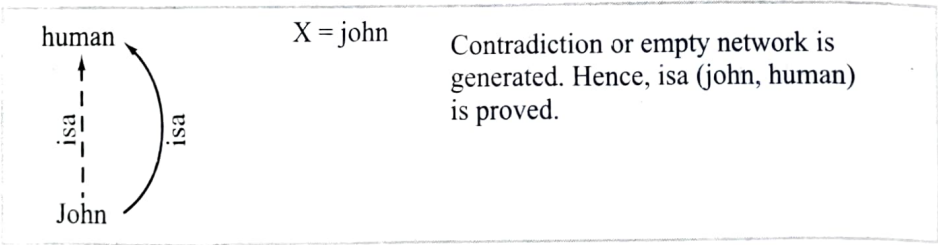
In backward inferencing mechanism, we can prove a conclusion or goal from a given ESNet by adding the denial of the conclusion to the network and show that the resulting set of clauses in the network gives contradiction. This is done by performing successive steps of resolution until an explicit contradiction is generated. It is similar to proof by resolution refutation in clausal form of logic. Consider the set of clauses (given in Table 7.5) represented in clausal form. We need to prove $isa(john, human)$ using the same network. The proof is shown in Table 7.5.

Table 7.5 Backward Reasoning Inference



After adding denial link in ESNet, we get the reduction in ESNet as shown in Table 7.6 by elimination of assertion and their denial link with the help of appropriate substitution.

Table 7.6 Reduction of ESNet



Thus, we can conclude that inferencing and inheritance are important and embedded capabilities of ESNet.

7.4.3 Example for Illustrating Inferencing Methods

Consider the following example, represented in clausal form:

- isa(X, living_thing) ← isa(X, animate)
- isa(X, animate) ← isa(X, human)
- isa(X, human) ← isa(X, man)
- isa(john, man)

The corresponding ESNet representation is shown in Fig. 7.10. Since the variables in different clauses are different, we choose different variables in ESNet to avoid confusion in the representation. Now, we proceed to illustrate both inferencing methods, namely, forward reasoning inference and backward reasoning inference, with the help of suitable examples.

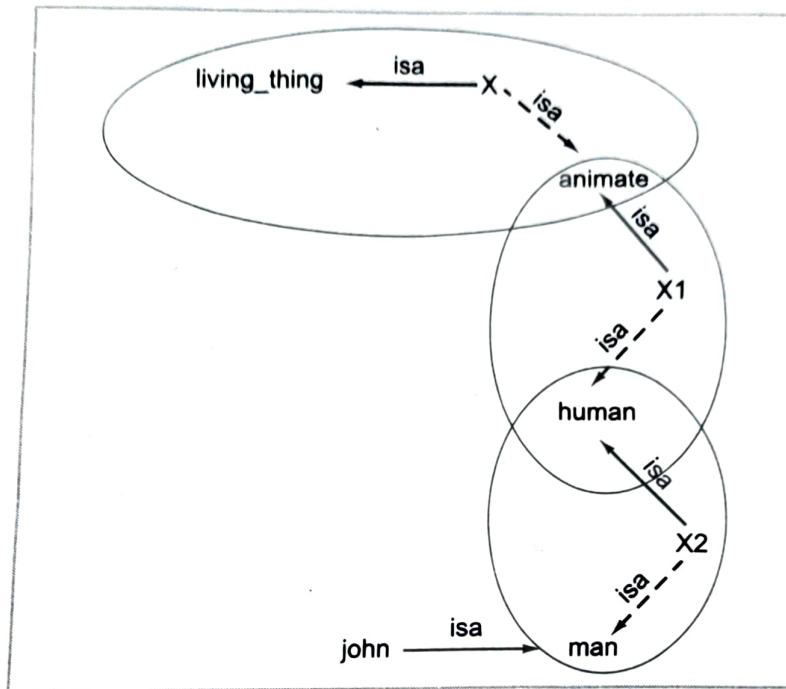
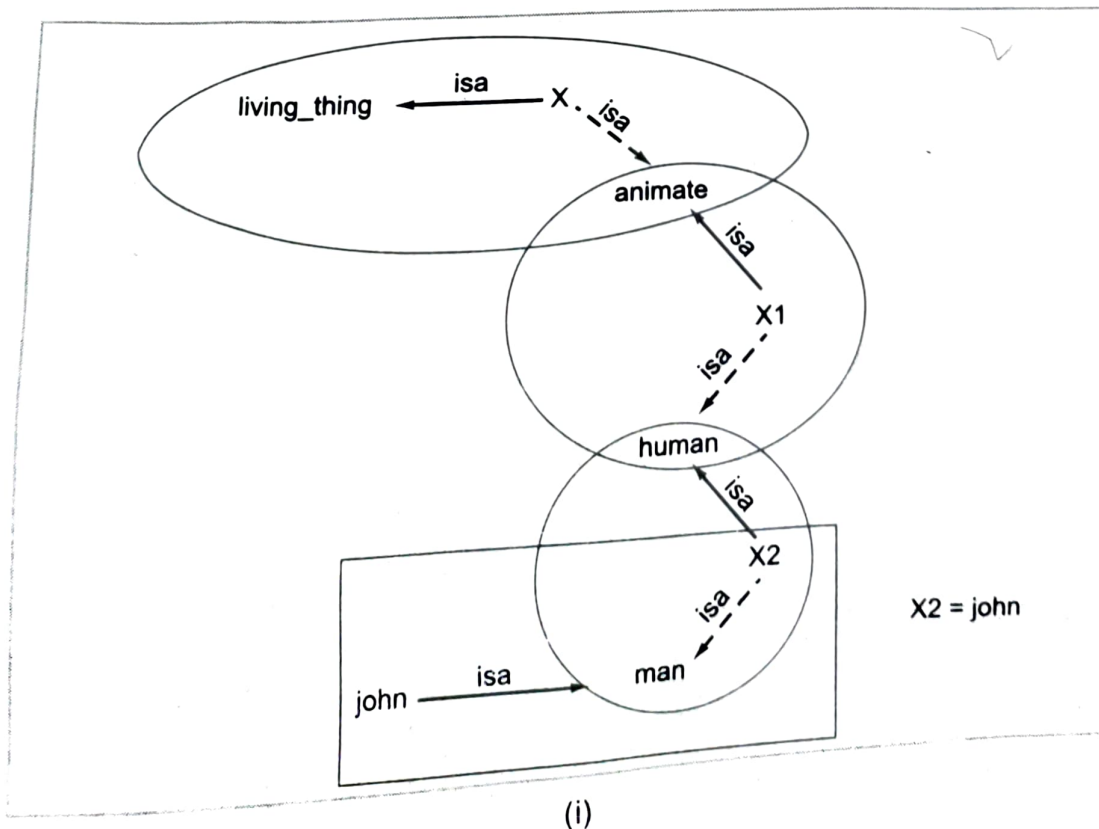
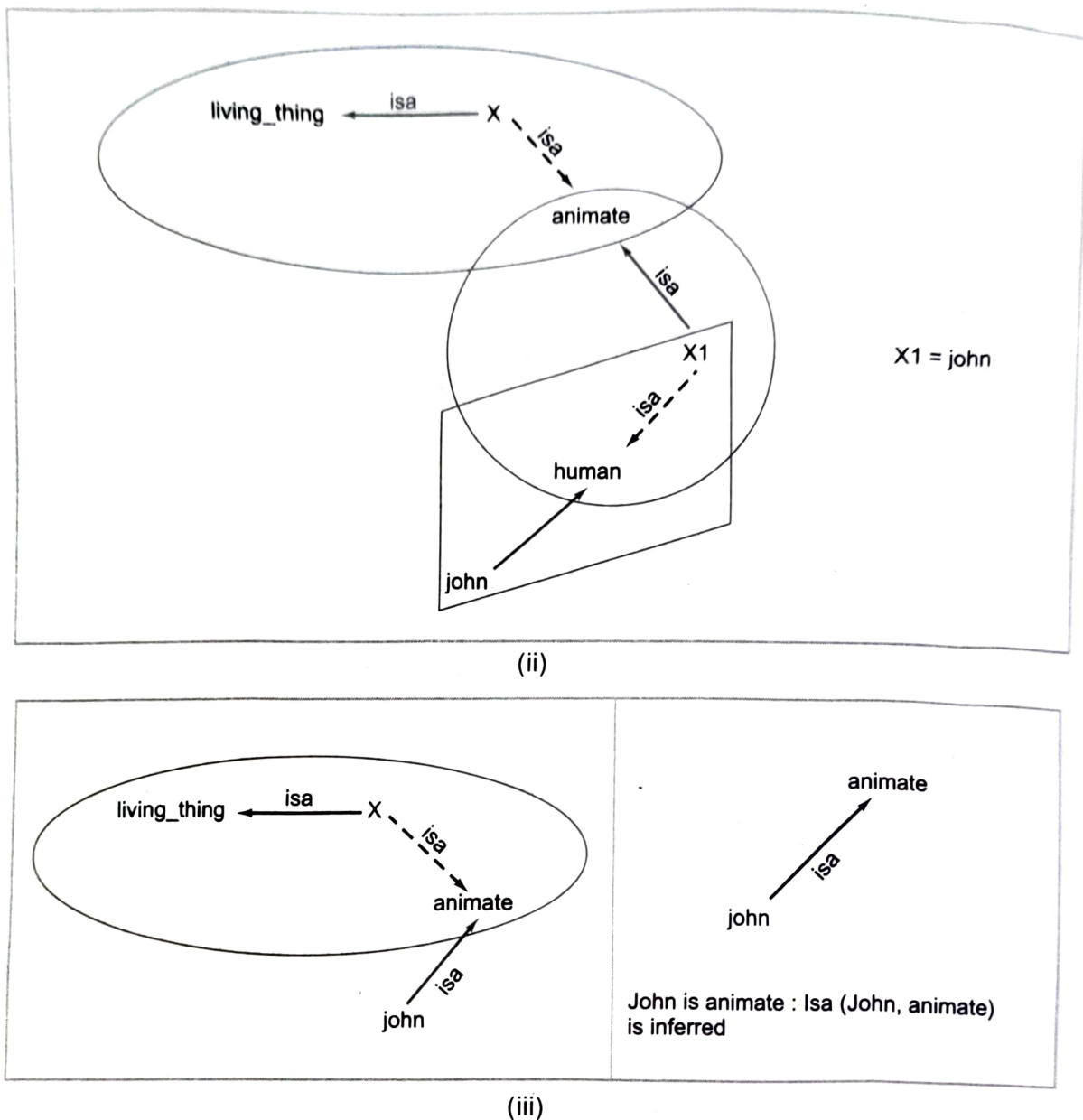


Figure 7.10 ESNet Representation

Forward Reasoning Inference

The new assertion *john is human* can be inferred or derived as discussed in previous example. The steps for inferring that *john is animate* are given in Fig. 7.11. The assertion *john is a living_thing* can be inferred similarly.

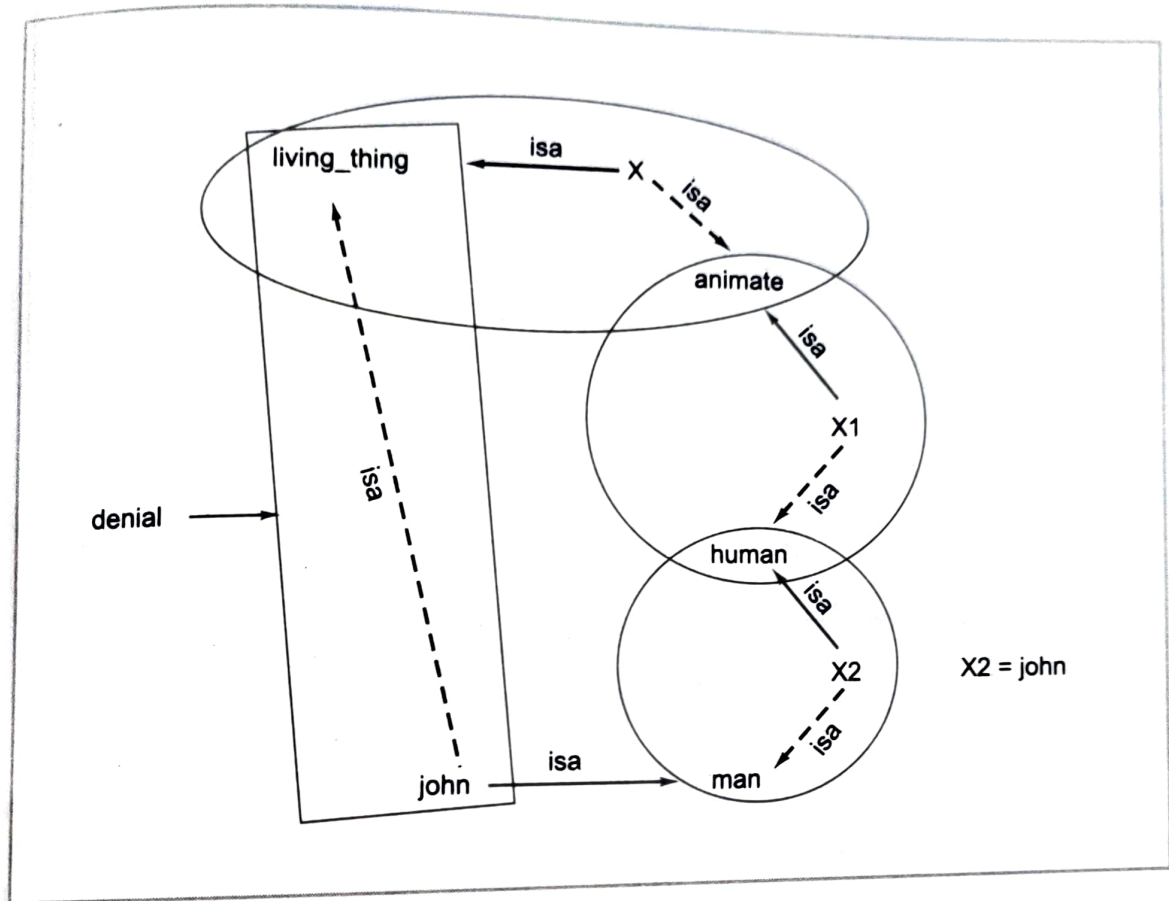


Figure 7.11 Derivation of *john is animate*

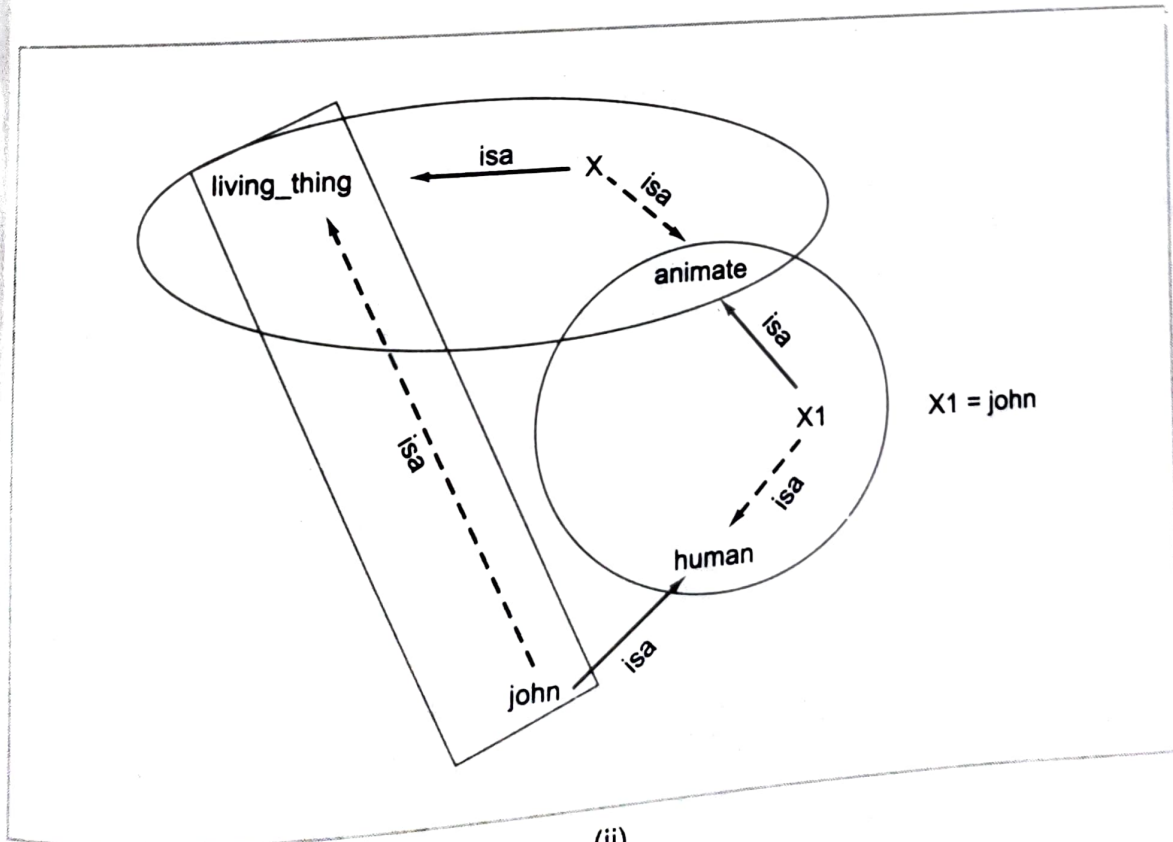
Further, John is `living_thing` can be derived by binding `X` with John.

Backward Reasoning Inference

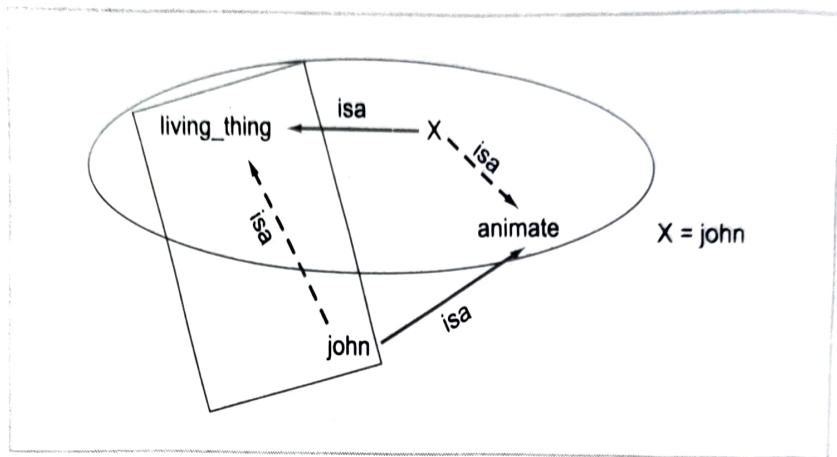
Let us solve a query `isa(john, living_thing)` (i.e., Is John a living thing?) using backward inferencing mechanism. In this mechanism, we add the denial of the fact that John is not a `living_thing` in ESNet and see if we can derive inconsistency or an empty network. In Fig. 7.12, each part shows reduction in the network by elimination of assertions and their denial links with the help of appropriate substitutions. Here denial links are enclosed in rectangular box to distinguish it from rest of network. Note that dotted lines are denial links, while continuous lines are assertion links. These are removed after substitution of actual value of a variable, if possible.



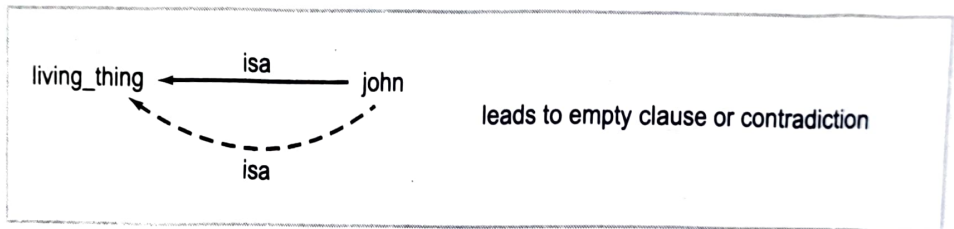
(i)



(ii)



(iii)



(iv)

Figure 7.12 Solving Query *isa(john, living_thing)*

Hence, we can prove the query that *john is an animate*.

Let us consider another example. The sentences, “*Anyone who gives something he likes to person likes that person also. John gives an apple to Mike. John likes an apple*”, can be expressed in both binary clausal form and ESNet representation as given below.

Clausal Representation

The clausal representation of the sentences given above may be written as

$\text{likes}(X, Z) \leftarrow \text{action}(E, \text{give}), \text{object}(E, Y), \text{actor}(E, X), \text{recipient}(E, Z), \text{likes}(X, Y)$

$\text{action}(e, \text{give}).$

$\text{object}(e, \text{apple}).$

$\text{actor}(e, \text{john}).$

$\text{recipient}(e, \text{mike}).$

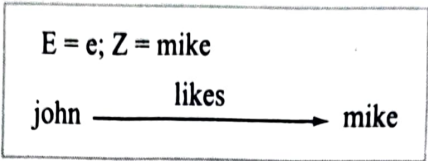
$\text{likes}(\text{john}, \text{apple}).$

Here, E is a variable which will be bound to an actual event ‘ e ’ in the process of resolution.

ESNet Representation

The extended semantic network for the clauses described above is given in Fig. 7.13.

The network shown in Fig. 7.14 is further reduced to the following conclusion by unifying $E = e$ and $Z = mike$.



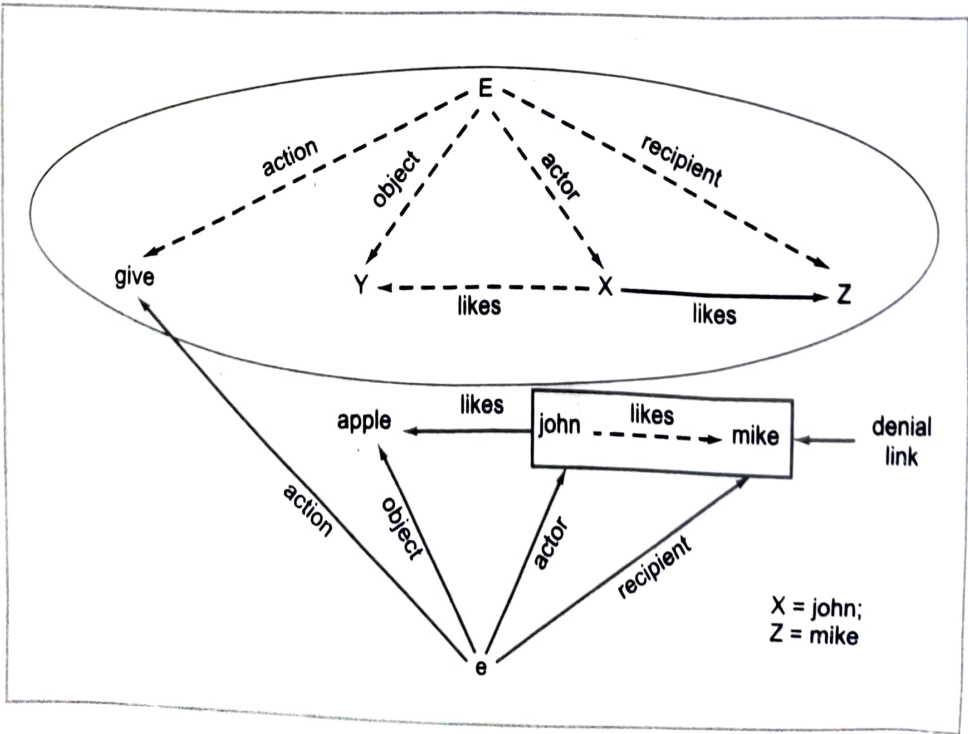
We can clearly see that a link *likes* between *john* and *mike* is deduced after eliminating all links which are assertions and denials. For example, *action(e, give)* is an assertion and *action(E, give)* is a denial, which is a contradiction and is hence eliminated. Similarly, other such eliminations take place and we are left with the final assertion *likes(john, mike)*.

Backward Reasoning Inference

Let us now prove *likes(john, mike)* using backward reasoning method. In order to prove *likes(john, mike)*, add the denial link to the network and try to reduce the network to empty. The denial link is shown as follows:



The reduction of the network is shown step by step in Fig. 7.15. Denial link is enclosed in rectangular box.



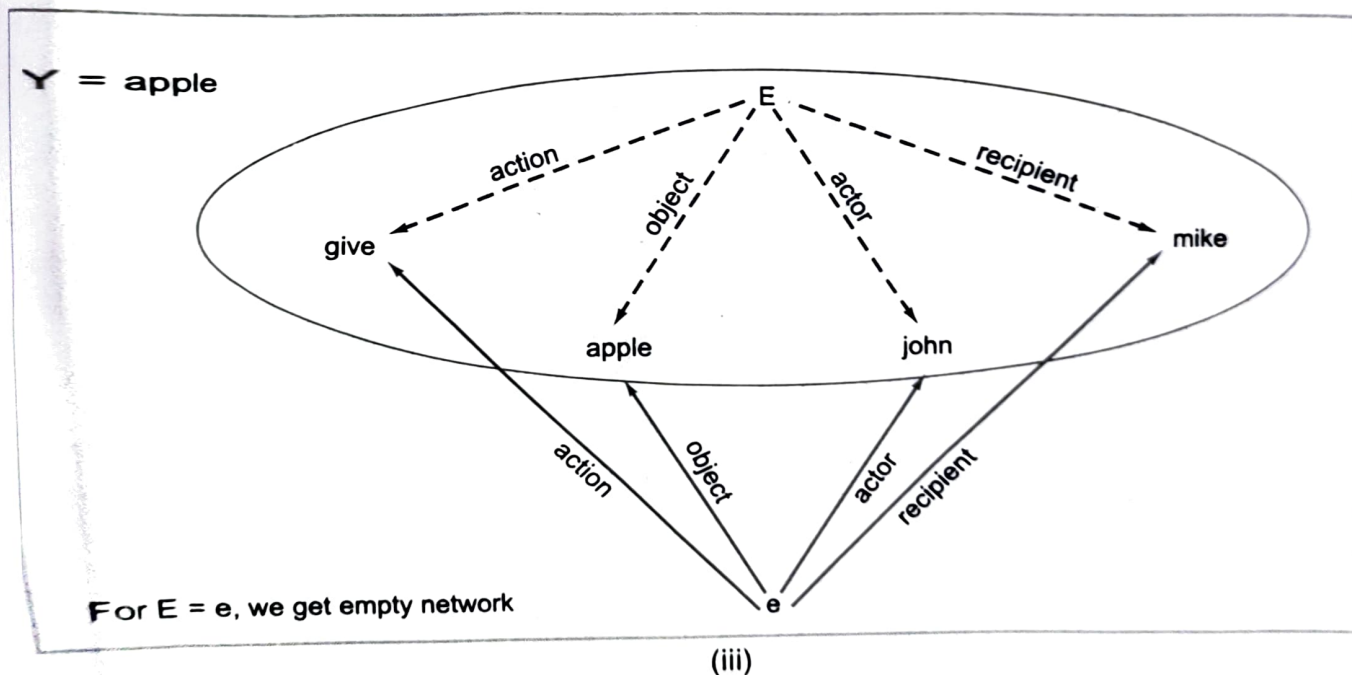
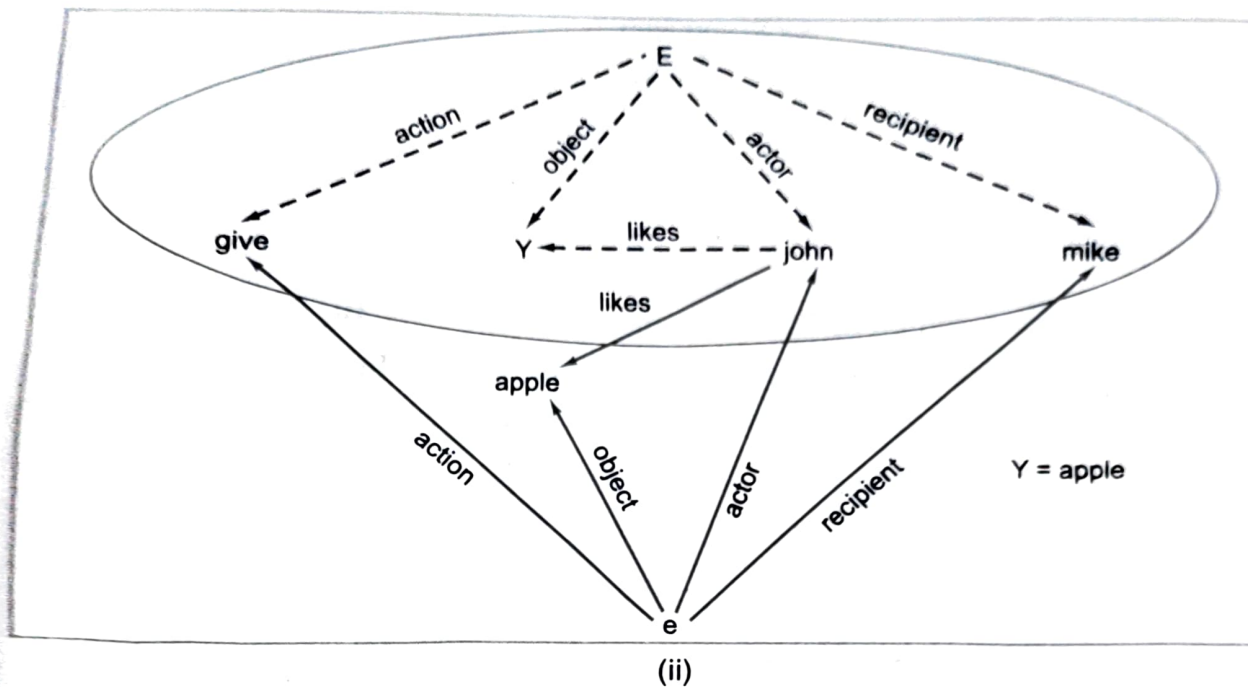


Figure 7.15 Reduction to Empty ESNet

When **E** is unified with 'e', then entire ESNet is reduced to contradiction and hence, the query *likes(john, mike)* is proved using backward reasoning inference process.

7.4.4 Inheritance

In a conventional semantic network, lower level nodes in *isa* hierarchy inherit properties from higher level nodes unless the properties are redefined in the node itself. Consider the following logic program:

In Fig. 7.18, we notice that *john is human who has two_legs* is an assertion and *john has two_legs* is a denial; so, we get a contradiction. Thus, we have proved *john has two_legs* using backward reasoning method.

7.4.5 Implementation

Implementation of ESNet can be done in any programming language or using a tool which facilitates implementation of semantic networks. The clauses can be represented either explicitly, by adding them to the network, or implicitly, by using the structure-sharing method [Boyer and Moore 1972]. The resolvents may be represented either explicitly or implicitly and by pointers to their parents along with a record of the matching substitution. When a resolvent is created, a parent may be deleted if no other match exists for the atom being matched in that parent. In practice, whenever deletion is possible, it is generally more convenient to construct the resolvent from the constituents of the deleted parent. Since deletion of clauses destroys the original network, it is advised to save a copy of the original network or restore it to its original form.

A major point of difference between conventional semantic networks and extended semantic networks is as follows: in a conventional semantic network, procedures are generally written in the host programming language, whereas in ESNet, procedures are integrated with the rest of the database and are executed by the same general-purpose mechanism which performs inference in the network.

Consider a sentence *john gives an apple to mike* and if we wish to ask *who takes an apple?*, then the knowledge representation scheme should have some mechanism of storing the knowledge that for every act of *giving*, there is a corresponding act of *taking*. Such rules can be easily integrated with the rest of the network and the information about its use is incorporated in the general-purpose inference system associated with ESNet. Such inferencing is not possible in conventional semantic networks or even in clausal logic unless we write explicit rule for it. The ESNet can also be considered as an abstract data structure for the implementation of a proof procedure. A characteristic feature of ESNet is indexing an argument or predicate where direct access is provided to all atoms (both conditions and conclusions) containing the given term or predicate. Note that an indexing scheme is a method of organizing information for efficient access of the same information at a later stage.

7.5 Knowledge Representation using Frames

Many of the ideas about frames and frame systems and how they can be used for the process of knowledge representation were first introduced by Marvin Minsky (1975). *Frames* are regarded as an extension to semantic nets; each node of a semantic net is represented by a frame. A *frame* may be defined as a data structure that is used for representing a stereotyped situation. It consists of a collection of attributes or slots and associated values that describe some real-world entity. There are several types of information attached to each frame. Although frames are not

particularly helpful on their own, frame systems form a powerful way of encoding information that supports reasoning. With the increase in complexity of a problem, the representation should become more structured for more beneficial use. Therefore, frames may be considered to represent the ways of organizing as well as packaging knowledge in a more structured form.

Frames are slightly similar to the concept of class of object-oriented paradigm; class also contains attributes and methods. In frames, it consists of *attributes* or *slots*; slots are described with attribute-value pairs <slot_name, value>. Slots are generally complex structures that have facets (or fillers) describing their properties. The value of a slot may be a primitive, such as a text string, constant or an integer, or it may be another frame. So, slots may contain *value*, refer to other frames (*relations*) or contain *methods*. Most of the frame systems allow multiple values for slots and some systems support procedural attachments as well. These procedural attachments (methods) can be used for computing the slot value whenever required. Frames may contain triggers for checking consistency or obtaining updates of other slots. Therefore, frames are basically a machine-usable formalization of concepts or schemata. The general structure of a frame is given in Table 7.7.

Table 7.7 Structure of a Frame

frame name
slot-filler
default values
constraints on values within the slots of a frame
pointers (links) to other frames
ako (a-kind-of or subclass)
inst (instance)
instantiation procedure
inheritance procedure
default inference procedure
triggers

A frame may contain as many <slots-filler> pairs as required to describe an object. Fillers are also known as facets. Each slot may contain one or more facets or fillers from the list given in Table 7.8.

Table 7.8 List of Facets in a Frame

Facet Name	Description
value	value of the slot
default	default value of the slot and it may be redefined by lower classes
range	the range of integer or enumerated values that a slot can have
demons	procedural attachments such as if_needed, if_deleted, if_added, etc.
other	may contain rules, other frames, semantic net, or any type of other information

A class frame generally has certain default values which can be redefined at lower levels. In case a class frame possesses an actual value facet, then decedent frames cannot modify that value; this value then remains unchanged for all subclasses and instances of that class. The related frames are linked together into *frame systems* and are organized into hierarchies or network of frames. Each frame in the network is either a class frame or an instance frame. As an example, a hospital frame has been defined in Table 7.9 (Bedi, 1999).

Table 7.9 Hospital Frame

Slot Name	Facet name	Facet value
F_name	value	Metro hospital
Country	default	India
Phone_No	default	23456778
Address	default	abc

Frames in a network of frames are connected using the links discussed below. Other required information may be made available using slot-value pair concept.

- **Ako:** This link connects two class frames, one of which is a kind of the other class, e.g., the class *child_hospital* is a kind of the class *hospital*. A class can define its own slots and also inherits slot-value pairs from its super class. It gives a sub-typing hierarchy where all instances of class frame are instances of super class frames. For example, all child hospitals are hospitals but all hospitals may not be child hospitals. With the help of this link, knowledge representation becomes more structured and memory efficient.
- **Inst:** This link connects a particular instance frame to a class frame, e.g., AIIMS is an instance of the class frame *hospital*. An instance class possesses the same structure as its class frame.
- **Part_of:** This link connects two class frames one of which is contained in the other class, e.g., *ward* is Part_of the class *hospital*.

We define a network of frame system for hospitals as described below.

Frame Descriptions

The following are descriptions of some frames of the network.

Hospital Frame (Root of the Network)

F_name: hospital
 Country: (value – India)
 Phone_No: (default – 23456778)
 Address: (default – New Delhi)
 Director: (default – xyz)

Labs: lab (Lab Frame)
Wards: ward (Ward Frame)
Doctors: doctor (Doctor Frame)

Child Hospital Frame

F_name: child_hospital
Ako: hospital (Hospital Frame)
Age: (range – [0–12]), (rule- *if_added*)

Heart Hospital Frame

F_name: heart_hospital
Ako: hospital (Hospital Frame)

Lab Frame

F_name: lab
Part_of: hospital (Hospital Frame)
Pathology: pathology (Pathology Frame)
X_Ray: x_ray (X_Ray Frame)

Ward Frame

F_name: ward
Part_of: hospital (Hospital Frame)
Ortho: orthopaedic (Orthopaedic Ward Frame)

Doctor Frame

F_name: doctor
Part_of: hospital
Qualification (default – MBBS)

Pathology Frame

F_name: pathology
Part_of : lab (Lab Frame)
Incharge: (default – xyz)
Types of tests: (....)

X_Ray Frame

F_name: x_ray

Part_of : lab (Lab Frame)
Incharge: (default – pqr)

Orthopaedic Ward Frame

F_name: orthopaedic
Ako : ward (Ward Frame)

Frame Instance Descriptions

Hospital Frame Instance

F_name: AIIMS
Inst: Hospital Frame
Phone_No: (value – 91 11 6591425)
Address: (value – Central_Delhi)
City: (value – New Delhi)
Director: (value – Dr. Smith)
Labs: (Lab-name)
Wards: Wards Frame Inst
Doctors: Doctors Frame Inst

Labs Frame Instance

F_name: lab-name
Inst: lab (Lab Frame)
Part_of: AIIMS (Hospital Frame Instance)
Pathology: Pathology Frame Instance
X_Ray: x_ray Frame Instance

Pathology Frame Instance

F_name: pathology-lab-name
Inst: pathology (Pathology Frame)
Part_of : lab-names (Labs Frame Inst)
Incharge: (value-Mr John)
Types of tests: (value- [ECG, Blood Test, Urine])

Child Hospital Frame Instance

F_name: kalawati
Inst: Child_hospital (Child Hospital Frame)

Address: (value- 1234 New Delhi)
Phone: (value-0116573891)
Director: Dr Mike

Heart Hospital Frame Instance

F_name: escorts
Inst: heart_hospital (Heart Hospital Frame)
Phone_No: (value-0114563732)
Address: (value-Faridabad)

The graphical representation of a frame network for the class 'hospital' described above is shown in Fig. 7.19.

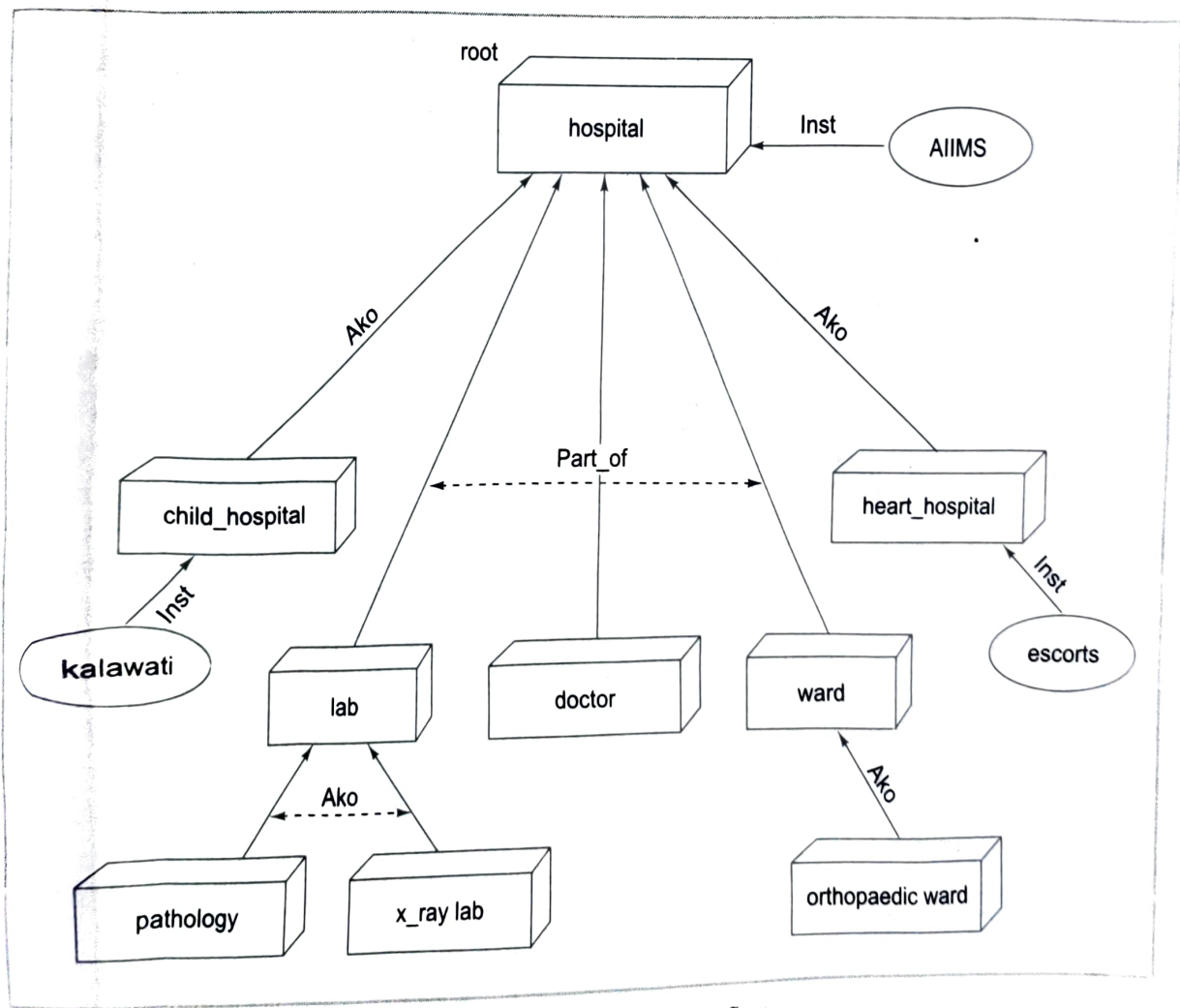


Figure 7.19 A Simple Frame System

A network of frames can be viewed as a three-dimensional figure: one dimension is concerned with the description of a class along with all the classes contained in it and connected with *Part_of*

link. The second dimension in the hierarchy is of *Ako* links, while the third dimension contains the instances of all classes connected with *Inst* links. It is important to note that each frame can have at most two links with the following combinations:

- A frame can be an instance of some frame and a part of another frame. So, it can have both *Inst* and *Part_of* links.
- A frame can be a-kind-of frame of one frame and a part of another frame. So, it can have *Ako* and *Part_of* links.
- However, it is not possible to have a frame which is both an instance and a-kind-of some class at the same time. So, the links *Inst* and *Ako* in a frame are not possible.

Figure 7.20 shows the three-dimensional view of the hospital frame structure.

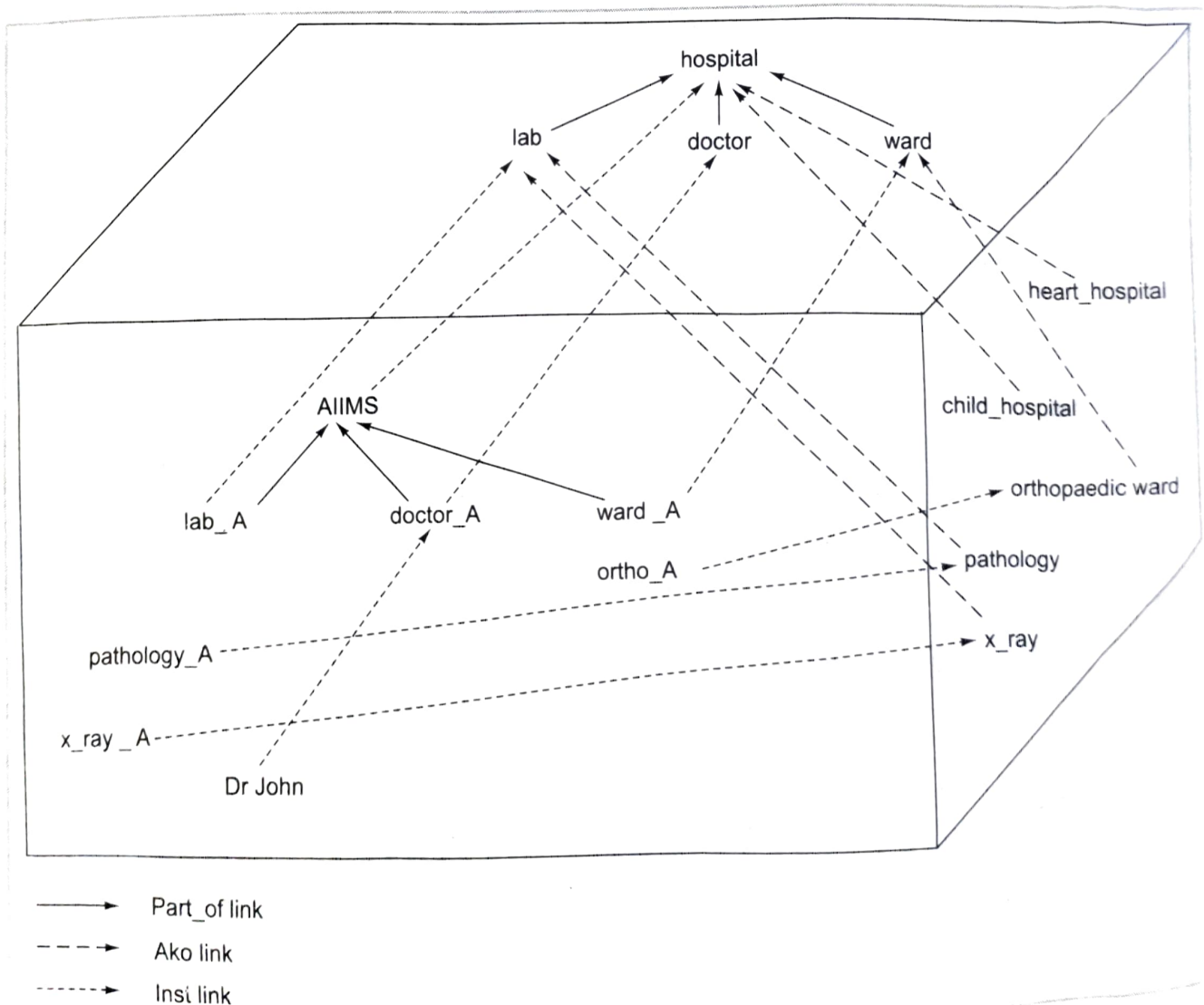


Figure 7.20 Three-dimensional View of the Class 'Hospital'

An instance of a hospital is a specific hospital, such as AIIMS, which will have instances of all the classes in the network of hospital frame, that is, instances of the *lab*, *doctor*, and *ward*.

Further, if we wish to represent the three-dimensional structure of a network of frames in first-order predicate logic [Chen, et al. 1986], it could be done as follows:

$$\forall X (\text{frame}(X) \equiv \exists Y_1, Y_2, \dots, Y_n ((\text{Ako}(X, Y_1) \vee \text{Inst}(X, Y_1)) \wedge \text{Part_of}(X, Y_2) \wedge \text{slot}_3(X, Y_3) \wedge \text{slot}_4(X, Y_4) \wedge \dots \wedge \text{slot}_n(X, Y_n))),$$

where, $\text{frame}(X)$ means that X is a frame, $\text{slot}_i \in \{\text{Inst}, \text{Ako}\}$; $\text{slot}_2 = \text{Part_of}$, and $\text{slot}_i(X, Y_i)$ means that Y_i is value of slot_i of frame X .

7.5.1 Inheritance in Frames

Inheritance is defined as a mechanism which is utilized for passing knowledge from one frame to other frames down through the taxonomy from general to specific frame. It is a good way of obtaining information that is not stored in the place we first looked. It leads to *cognitive economy*, where information is only stored in one place, while it can be retrieved from different parts of the network.

Attachment of Demons (Rules)

Demons allow us to invoke rules within the frames and are considered to be a powerful style of programming. These can be attached with a slot along with other required facets. For example, an attached demon *if_needed* may be used for monitoring the behaviour of the system. Another demon *if_added* may be used to validate data when the data is added in the value slot. For example, if value of slot *age* is to be entered for a patient of child hospital, it will check whether it lies in the specified range.

Another use of attaching demons is that they allow dynamic information retrieval and storage. Here, one may get the value directly from the slot or may have to dynamically calculate the value of the required slot on the basis of other values. For example, the doctor frame can have a slot for tax that contains a demon *if_needed* for calculating tax using salary information available with each doctor instances. There is no need to store the tax calculated for each doctor separately, instead it can be calculated using the demon stored in doctor frame at the tax slot. Let us consider another example. If date_of_birth slot for a doctor is given and we are interested in knowing his/her age, a demon that calculates age from date-of-birth value and the current date (which can be obtained from the system) could be easily calculated.

7.5.2 Implementation of Frame Knowledge

Frames can be easily implemented using object-oriented programming (OOP) languages. The concept of class in OOP can be directly used to code frames that has the concept of subclass(Ako) and containment (Part_of link) of a class within another class. Instances of the frames can be declared as objects of the relevant classes. Slots can be defined as variables and procedural attachment as methods. Inheritance is inbuilt in these languages.

Representation of Frames in Prolog Frames can also be easily implemented using Prolog language. Each frame is represented as a fact in Prolog as *frame(F_name, [slot₁(facet(value), ...), slot₂(facet(value), ...), ...])*. Let us represent part of the network of frame system for hospital in Prolog as follows.

```
frame(hospital,[nil, country(default(india)), phone(default(9111234567)),
               address(nil),labs(labs), wards(ward), doctors(doctor)]]).
frame(child_hospital,[ako(hospital), age(value(0,12), demon(if_needed))]).
frame(heart_hospital,[ako(hospital)]).
frame(labs, [part_of(hospital), pathology(pathology), x_ray(x_ray)]).
frame(aiims, [inst(hospital), phone(value(91116591425)), address(value(central_
               delhi)), city(value(delhi)), labs(labs_inst),
               wards(ward_inst), doctors([list of doctors])]).
frame(labs_inst, [inst(labs), part_of(aiims), pathology(pathology_inst),
                  x_ray(x_ray_inst)] ).
frame(pathology_inst, [inst(pathology), incharge(value(pqr)), test([ecg, blood_test,
                  urine])] ).
frame(kalawati, [inst(child_hospital), address(value("1234 new delhi")),
                  phone(value(0116573891)), director(value(Dr Mike))].
frame(escort, [inst(heart_hospital), phone(value(0114563732)),
               address(value(faridabad))].
```

Inheritance Rules in Prolog

Since we have coded network of frames in Prolog, we need to write inheritance rules as well to obtain relevant information either directly available in the frame or through inheritance. Suppose we wish to know the country of 'escort hospital', which is not given in this frame directly. We can travel starting from escort upward till root to know that country of escort hospital is India. An inheritance rules for frames might look like the one given below.

```
find(X, Y) :- frame(X, Z), search(Y, Z), !.
find(X, Y) :- frame(X, [inst(Z)|_]), find(Y, Z), !.
find(X, Y) :- frame(X, [ako(Z)|_]), find(Y, Z), !.
find(X, Y) :- frame(X, [part_of(Z)|_]), find(Y, Z).
```

Predicate search will basically retrieve the list of slots–facet pair and will try to match Y for slot. If a match is found then its facet value is retrieved, otherwise the process is continued till we reach the root frame.

Exercises

7.1 Draw a semantic network representing the following knowledge:

- i. Every vehicle is a physical object. Every car is a vehicle. Every car has four wheels. Electrical system is a part of car. Battery is a part of electrical system. Pollution system is a part of every vehicle. Vehicle is used in transportation. Suzuki is a car.
- ii. Every living thing needs oxygen to live. Every human is a living thing. John is human. Answer the query John is a living thing and John needs oxygen to live using inheritance.

7.2 Write an inheritance rules in Prolog to answer queries related to the statements given in part (i) of Problem 7.1, such as Suzuki has battery, electrical system, pollution system, etc.

7.3 Draw an extended semantic network for representing the following English text and infer the conclusion for each of the following:

- i. Teachers who work hard are liked by students. Mary is a hardworking teacher. John is a student. Conclude that John likes Mary.
- ii. Everyone who sees a movie in a theatre has to buy a ticket. A person who does not have money cannot buy a ticket. John sees a movie. Conclude that John had money.
- iii. All senior citizens and politicians get air concession. Mary is a senior citizen. John does not get air concession. Conclude that Mary gets air concession and John is neither senior citizen nor politician.
- iv. Every member of an association named ROA is either retired or an officer of central government. John is a member of ROA. Conclude John is retired or an officer.

7.4 Develop a complete Frame-Based System (FBS) for University and Hospital applications in Prolog.

7.5 Create a network of frames (NOF) with `ako`, `a_part_of`, and `inst` links with the following characteristics:

- i. Insert a frame in NOF with all slot values filled up.
- ii. Delete a frame from NOF.
- iii. Update the value of the slot of a given frame.
- iv. Query module to ask questions using FBS.